

Práctica Programación Dinámica

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Repaso



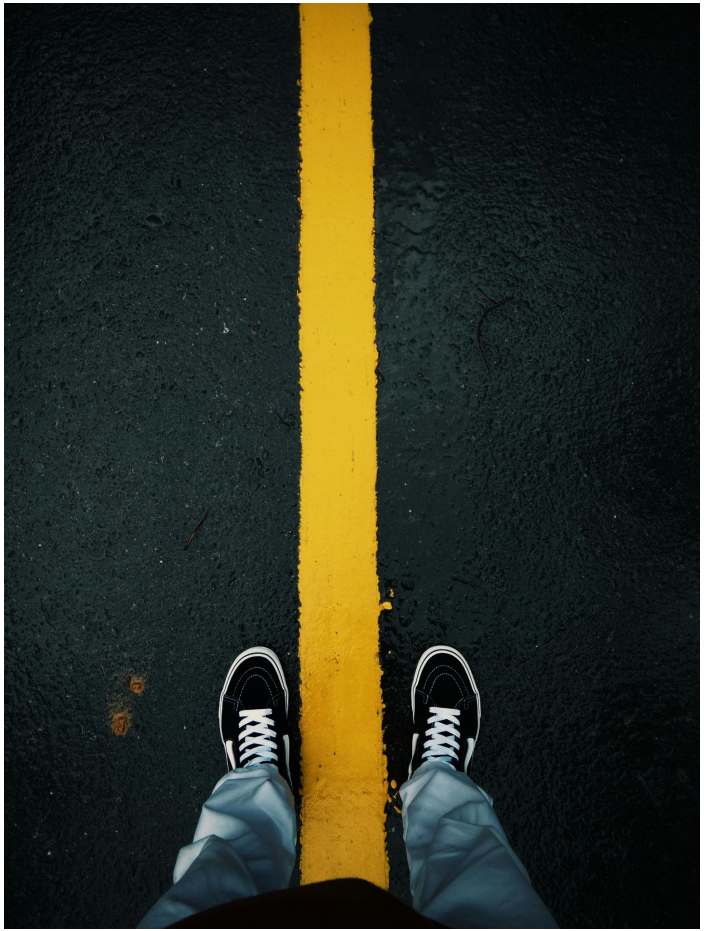
Cottonbro Studio

Repaso

Programación Dinámica

- Es otra técnica más de diseño de algoritmos
  - Programación: en el sentido de planificación, toma de decisiones
  - Dinámica: en el sentido de “múltiples etapas”
- Muy buena para problemas de optimización
- Enfoque Bottom-Up (en contraste con D&C que es Top-Down)
- Requisitos:
  - Subestructura Óptima: la solución óptima se puede construir a partir de las soluciones óptimas de los subproblemas
  - Subproblemas Superpuestos: Resolver un subproblema requiere la solución de otros subproblemas más chicos resueltos previamente
- Memoization

Ejercicio #2 (Buchwald)



Muhammad Ghazie

Ejercicio #2 (Buchwald)

Dada un aula/sala donde se pueden dar charlas. Las charlas tienen horario de inicio y fin. Además, cada charla tiene asociado un valor de ganancia. Implementar un algoritmo que, utilizando programación dinámica, reciba un arreglo que en cada posición tenga una charla representada por una tripla de *inicio*, *fin* y *valor* de cada charla, e indique cuáles son las charlas a dar para maximizar la ganancia total obtenida. Indicar y justificar la complejidad del algoritmo implementado.

Ejercicio #2 (Buchwald)

- ¿Se parece a algo que conozcamos?
  - Weighted Interval Scheduling
- ¿Cómo se puede aprovechar el arreglo con las triplas?
- ¿Hacemos un trabajo previo con las triplas?
- ¿Cómo hacemos para averiguar si dos charlas se solapan?
- ¿Cómo hacemos para averiguar cuál es la última charla que se solapa con k-ésima?
- En el paso a paso, ¿qué decisiones hay que tomar?

Ejercicio #2 (Buchwald)

Plan de acción:

- Ordenamos las triplas por menor horario de fin
- P(k): devuelve el índice de la tarea más tardía que no se solapa con k, o 0 si se solapa
- Veamos las decisiones posibles en cada paso:
  - Incluir la tarea k: tomo el óptimo para la tarea P(k) y le sumo v_k
  - No incluir la tarea k: me quedo con el óptimo para la tarea k-1

Relación de Recurrencia:

- $OPT(k)=max(\ v_k + OPT(P(k));\ \ OPT(k-1)\ )$

Ejercicio #14 (Buchwald)



Vika Glitter

Ejercicio #14 (Buchwald)

Somos ayudantes del gran ladrón el Lunático, que está pensando en su próximo atraco. Decidió en este caso robar toda una calle en un barrio privado, que tiene la particularidad de ser circular. Sabemos cuánto se puede obtener por robar en cada casa. Podemos enumerar a la primer casa como la casa 0, de la cual podríamos obtener g_0, la casa a su derecha es la 1, que nos daría g_1, y así hasta llegar a la casa n-1, que nos daría g_{n-1}. Toda casa i se considera adyacente a las casas i-1 e i+1. Además, como la calle es circular, la casas 0 y n−1 también son vecinas. El problema con el que cuenta el Lunático es que sabe de experiencias anteriores que, si roba en una casa, los vecinos directos se enterarían muy rápido. No le daría tiempo a luego intentar robarles a ellos. Es decir, para robar una casa debe prescindir de robarle a sus vecinos directos.

Ejercicio #14 (Buchwald)

El Lunático nos encarga saber cuáles casas debería atracar y cuál sería la ganancia máxima obtenible. Dado que nosotros nos llevamos un porcentaje de dicha ganancia, vamos a buscar el óptimo de este problema.
Implementar un algoritmo que, por programación dinámica, obtenga la ganancia óptima, así como cuáles casas habría que robar, a partir de recibir un arreglo de las ganancias obtenibles. Para esto, escribir y describir la ecuación de recurrencia correspondiente. Indicar y justificar la complejidad del algoritmo propuesto.

Ejercicio #14 (Buchwald)

- ¿Se parece a algo que conozcamos?
  - Problema de las Materias
- ¿Hacemos un trabajo previo con el arreglo de ganancias?
- ¿Cómo vamos a manejar el solapamiento?
- En el paso a paso, ¿qué decisiones hay que tomar?

Ejercicio #14 (Buchwald)

Plan de acción:

- Veamos las decisiones posibles en cada paso:
  - Robar la casa k: tomo el óptimo para la casa k-2 y le sumo g_k
  - No robar la casa k: me quedo con el óptimo para la casa k-1
- Caso base:
  - Robar de la casa 0 hasta la casa n-2 (se excluye la casa n-1)
  - Robar de la casa 1 hasta la casa n-1 (se excluye la casa 0)

Relación de Recurrencia:

- $OPT(k)=\max(\ g_k + OPT(k-2);\quad OPT(k-1)\ )$

Ejercicio #14 (Buchwald)

Relación de Recurrencia:

- $OPT(k)=max(\ \ g_k + OPT(k-2);\ \ \ OPT(k-1)\ \ )$

Caso Base:

- $OPT(0)=g_0$
- $OPT(1)=max(\ \ g_0\ ;\ \ \ g_1\ )$

Ejercicio #15 (Buchwald)



Brian Lazo

Ejercicio #15 (Buchwald)

Dada una soga de n metros ($n \geq 2$), implementar un algoritmo que, utilizando programación dinámica, permita cortarla (en partes de largo entero) de manera tal que el producto de los largos de cada una de las partes resultantes sea máximo. El algoritmo debe devolver el valor del producto máximo alcanzable. Tener en cuenta que la soga puede cortarse varias veces. Indicar y justificar la complejidad del algoritmo.

n = 2 –> Debe devolver 2 (producto máximo es 2)
n = 3 –> Debe devolver 3 (producto máximo es 3)
n = 4 –> Debe devolver 4 (producto máximo es 2 * 2)
n = 5 –> Debe devolver 6 (producto máximo es 2 * 3)
n = 6 –> Debe devolver 9 (producto máximo es 3 * 3)
n = 10 -> Debe devolver 36 (producto máximo es 3 * 3 * 4)

Ejercicio #15 (Buchwald)

- ¿Se parece a algo que conozcamos?
  - Corte de la varilla?
- Juguemos con los ejemplos
- En el paso a paso, ¿qué decisiones hay que tomar?

Ejercicio #15 (Buchwald)

Plan de acción:

- Veamos las decisiones posibles para una longitud n:
  - No corto: n
  - Corto con k=1: Lo mejor de (1) * Lo mejor de (n-1)
  - Corto con k=2: Lo mejor de (2) * Lo mejor de (n-2)
  - Corto con k=3: Lo mejor de (3) * Lo mejor de (n-3)
  - ...
  - Corto con k=n/2: Lo mejor de (n/2) * Lo mejor de (n/2)

Relación de Recurrencia:

$$OPT(n)=max(OPT(1)*OPT(n-1);\ OPT(2)*OPT(n-2);\ ...;\ OPT(n/2)*OPT(n/2)\ )$$

Ejercicio #15 (Buchwald)

Casos base, teniendo en cuenta que la soga debe ser cortada al menos una vez:

- $OPT(0) = 0$ *(opcional)*
- $OPT(2) = 1$
- $OPT(3) = 2$

Ecuación de recurrencia:

- $OPT(n) = max\{ i \cdot max[(n-i), OPT(n-i)]\}$, con $1 \leq i < n$

Ejercicio #15 (Buchwald)

Seguimiento para n=10

i= 1 → 1 · max(9, 27) = 27

i = 2 → 2 ·max(8, 18) = 36

i = 3 → 3 · max(7, 12) = 36

i = 4 → 4 · max(6, 9) = 36

i = 5 → 5 · max(5, 6) = 30

i = 6 → 6 · max(4, 4) = 24

i = 7 → 7 · max(3, 2) = 21

i = 8 → 8 · max(2, 1) = 16

i = 9 → 9 · max(1, 0) = 9

→ OPT[10] = max(0, 27, 36, 36, 36, 30, 24, 21, 16, 9) = 36



¡Muchas gracias!